cadence®

Cadence SKILL++ Object System Reference

Product Version 6.1.6
November 2014

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Contents

Preface

This manual introduces the SKILL++ language to new users, leads users to understand advanced topics and encourages sound SKILL programming methods. It is aimed at the following users:

- CAD developers who have experience programming in SKILL, both Cadence internal users and Cadence® customers
- Programmers beginning to program in SKILL++
- CAD integrators

Licensing in Cadence SKILL Language

Product license 111 is checked out at `skill` executable startup or at workbench startup.

For information on licensing in the Cadence SKILL Language, see *Virtuoso Software Licensing and Configuration Guide.*

About the SKILL Language

The SKILL programming language lets you customize and extend your design environment. SKILL provides a safe, high-level programming environment that automatically handles many traditional system programming operations, such as memory management. SKILL programs can be immediately executed in the Cadence environment.

SKILL is ideal for rapid prototyping. You can incrementally validate the steps of your algorithm before incorporating them in a larger program.

Storage management errors are persistently the most common reason cited for schedule delays in traditional software development. SKILL's automatic storage management relieves your program of the burden of explicit storage management. You gain control of your software development schedule.

SKILL also controls notoriously error-prone system programming tasks like list management and complex exception handling, allowing you to focus on the relevant details of your

algorithm or user interface design. Your programs will be more maintainable because they will be more concise.

The Cadence environment allows SKILL program development such as user interface customization. The SKILL Development Environment contains powerful tracing, debugging, and profiling tools for more ambitious projects.

SKILL leverages your investment in Cadence technology because you can combine existing functionality and add new capabilities.

SKILL allows you to access and control all the components of your tool environment: the User Interface Management System, the Design Database, and the commands of any integrated design tool. You can even loosely couple proprietary design tools as separate processes with SKILL's interprocess communication facilities.

Related Documents

The following documents provide more information about SKILL and other topics discussed in this guide.

Installation, Environment, and Infrastructure

- For more information on installing Cadence products, see the *Cadence Installation Guide*.
- For information on database SKILL functions, including data access functions, see the see the *Virtuoso Design Environment SKILL Reference*. It contains APIs for the graphics editor, database access, design management, technology file administration, online environment, design flow, user entry, display lists, component description format, and graph browser.
- *Cadence User Interface SKILL Reference* contains APIs for management of windows and forms.
- *Virtuoso Software Licensing and Configuration Guide* contains SKILL licensing functions.

Section Names and Meanings

Each function described in this book can have up to seven sections. Not every section is required for every function description.

`Syntax`	The syntax requirements for this function.
`Prerequisites`	Steps required before calling this function.
`Description`	A brief phrase identifying the purpose of the function and the text description of the operation performed by the function.
`Arguments`	An explanation of the arguments input to the function.
`Return Value`	An explanation of the value returned by the function.
`Example`	Actual SKILL code using this function.
`References`	Other functions that are relevant to the operation of this function: ones with partial or similar functionality or which could be called by or could call this function. Sections in this manual which explain how to use this function.

Typographic and Syntax Conventions

The following typographic and syntax conventions are used in this document.

`literal (LITERAL)`
: Nonitalic (UPPERCASE) words indicate keywords that you must enter literally. These keywords represent command (function, routine) or option names.

`argument (z_argument)`
: Words in italics indicate text that you must replace with an appropriate argument. The prefix (in this case, z_) indicates the data type that the argument can accept. Names are case sensitive. Do not type the data type or underscore before your arguments.

`|`
: Vertical bars (OR-bars) separate the choice of options. They take precedence over any other character.

`[ ]`
: Brackets denote optional arguments. When used with vertical bars, they enclose a list of choices from which you can choose one.

`{ }`	Braces are used with vertical bars and enclose a list of choices from which you must choose one.

`...`	Three dots (`...`) indicate that you can repeat the previous argument. If you use them with brackets, you can specify zero or more arguments. If they are used without brackets, you must specify at least one argument, but you can specify more.
	`argument...  ;specify at least one,`
	`;but more are possible`
	`[argument]... ;specify zero or more`

`,...`	A comma and three dots together indicate that if you specify more than one argument, you must separate those arguments by commas.
`=>`	A right arrow points to the return values of the function. Variable values returned by the software are shown in italics. Returned literals, such as `t` and `nil`, are in plain text. The right arrow is also used in code examples in SKILL manuals.
`/`	Separates the possible values that can be returned by a Cadence SKILL language function.
`text`	Indicates names of manuals, menu commands, form buttons, and form fields.

SKILL Syntax Examples

The following examples show typical syntax characters used in SKILL.

Example 1

```
list( g_arg1 [g_arg2] ...) => l_result
```

This example illustrates the following syntax characters.

`list`	Plain type indicates words that you must enter literally.
`g_arg1`	Words in italics indicate arguments for which you must substitute a name or a value.

`( )`	Parentheses separate names of functions from their arguments.
`_`	An underscore separates an argument type (left) from an argument name (right).
`[ ]`	Brackets indicate that the enclosed argument is optional.
`...`	Three dots indicate that the preceding item can appear any number of times.
`=>`	A right arrow points to the description of the return value of the function. Also used in code examples in SKILL manuals.
`l_result`	All SKILL functions compute a data value known as the return value of the function.

Example 2

```
needNCells( s_cellType | st_userType x_cellCount) => t / nil
```

This example illustrates two additional syntax characters.

`|`	Vertical bars separate a choice of required options.
`/`	Slashes separate possible return values.

Identifiers Used to Denote Data Types

The Cadence SKILL language supports several data types to identify the type of value you can assign to an argument.

Data types are identified by a single letter followed by an underscore; for example, *t* is the data type in *t_viewNames*. Data types and the underscore are used as identifiers only; they should not be typed.

Prefix	Internal Name	Data Type
a	array	array
A	amsobject	AMS object
b	ddUserType	DDPI object

Prefix	Internal Name	Data Type
B	ddCatUserType	DDPI category object
C	opfcontext	OPF context
d	dbobject	Cadence database object (CDBA)
e	envobj	environment
f	flonum	floating-point number
F	opffile	OPF file ID
g	general	any data type
G	gdmSpecIlUserType	gdm spec
h	hdbobject	hierarchical database configuration object
K	mapiobject	MAPI object
l	list	linked list
L	tc	Technology file time stamp
m	nmpIlUserType	nmpIl user type
M	cdsEvalObject	—
n	number	integer or floating-point number
o	userType	user-defined type (other)
p	port	I/O port
q	gdmspecListIlUserType	gdm spec list
r	defstruct	defstruct
R	rodObj	relative object design (ROD) object
s	symbol	symbol
S	stringSymbol	symbol or character string
t	string	character string (text)
T	txobject	Transient object
u	function	function object, either the name of a function (symbol) or a lambda function body (list)
U	funobj	function object
v	hdbpath	—

Prefix	Internal Name	Data Type
`w`	wtype	window type
`x`	integer	integer number
`y`	binary	binary function
`&`	pointer	pointer type

For information on SKILL language, see *Cadence SKILL Language User Guide*.

Additional Learning Resources

Cadence offers the following training courses on the SKILL programming language:

- SKILL Language Programming Introduction
- SKILL Language Programming
- Advanced SKILL Language Programming

For further information on these and other related Virtuoso Layout Suite training courses available in your region, visit the Cadence Training portal. You can also write to training_enroll@cadence.com.

Note: The links in this section open in a new browser. The course links initially display the requested training information for North America, but if required, you can navigate to the courses available in other regions

How to Contact Technical Support

Cadence Customer Support is region specific. To find out the e-mail address, phone number, or fax number for your region, select the *Contacts* button from the *Cadence Online Support* home page (http://support.cadence.com).

You can also use *Cadence Online Support* to find out the latest information on Virtuoso Parasitic Aware Design.

1

Classes and Instances

allocateInstance

```
allocateInstance(
     u_class
     )
     => g_instance
```

Description

Creates and returns an empty instance of the specified class. All slots of the new instance are unbound.

Arguments

u_class	The specified class for which a new instance has to be created.

Value Returned

g_instance	An empty instance of `u_class`.

Example

```
defclass(A () ((slot1 @initform 1) (slot2 @initform 2)))
i = allocateInstance(findClass('A))
i->??
=> (slot1 \*slotUnbound\* slot2 \*slotUnbound\*)
i = allocateInstance('A)
i->??
=> (slot1 \*slotUnbound\* slot2 \*slotUnbound\*)
```

changeClass

```
changeClass(
     g_inst
     g_className
     [g_initArgs]
     )
     => g_updatedInst
```

Description

Changes the class of the given instance (`g_inst`) to the specified class (`g_className`).

The function `updateInstanceForDifferentClass()` is called on the modified instance to allow applications to deal with new or lost slots.

Arguments

g_inst	An instance of standardObject.
g_className	The new class for the instance.
g_initArgs	Additional arguments to be passed to the `updateInstanceForDifferentClass()` function.

Value Returned

`g_updatedInst`	The updated instance.

Example

```
(defclass A () ())
(defclass B (A) ((slot @initarg s)))
    x = (makeInstance 'A)
(changeClass x 'B ?s 1)
(classOf x)
    => class:B
```

className

```
className(
     us_class
     )
     => s_className
```

Description

Returns the class symbol denoting a class object.

For user-defined classes, *s_className* is the symbol passed to `defclass` in defining *us_class*.

Arguments

us_class	Must be a class object or a class symbol. Otherwise an error is signaled.

Value Returned

s_className	The class symbol.

Example

```
className( classOf( 5 )) => fixnum

defclass( GeometricObject
    ()      ;;; standardObject is the subclass by default
    ()      ;;; no slots
    )       ; defclass
className(findClass( 'GeometricObject))
=> GeometricObject

geom = makeInstance( 'GeometricObject )
className( classOf( geom))
=> GeometricObject
```

Reference

classOf, findClass

classOf

```
classOf(
     g_object
     )
     => u_classObject
```

Description

Returns the class object of which the given object is an instance.

Arguments

g_object	Any SKILL object.

Value Returned

u_classObject	Class object of which the given object is an instance.

Examples

```
classOf( 5 )
=> class:fixnum

className( classOf( 5 ))
=> fixnum
```

Reference

className, findClass

classp

```
classp(
     g_object
     su_class
     )
     => t | nil
```

Description

Checks if the given object is an instance of the given class or is an instance of one of its subclasses.

Arguments

g_object	Any SKILL object
su_class	A class object or a symbol denoting a class.

Value Returned

`t`	If the given object is an instance of the class or a subclass of the class.
`nil`	Otherwise.

Example

```
classp( 5 classOf( 5 ))  => t
classp( 5 'fixnum )      => t
classp( 5 'string )      => nil
classp( 5 'noClass )
*Error* classp: second argument must be a class - nil
```

Reference

classOf, className

defclass

```
defclass(
     s_className
          ( [ s_superClassName ] )
          ( [ ( s_slotName
          [ @initarg s_argName ]
          [ @reader s_readerFun ]
          [ @writer s_writerFun ]
          [ @initform g_exp ])
          ... ] )
     )
     => t
```

Description

Creates a class object with class name and optional super class name and slot specifications. This is a macro form.

If the super class is not given, the default super class is the `standardObject` class. Each slot specifier itself is a list composed of slot options. The only required slot option is the slot name.

Note: If you define a class with two slots that have the same name, as shown in the example given below, SKILL creates the class but also issues a warning.

```
defclass(A () ((slotA) (slotB) (slotA @initform 42)))
```

Arguments

s_className	Name of new class.
s_superClassName	Name of super class. Default is `standardObject`.
s_slotName	Name of the slot.
`@initarg` *s_argName*	Declares an initialization argument named *s_argName*. Calls to `makeInstance` can use *s_argName* as keyword argument to pass an initialization value.
`@reader` *s_readerFun*	Specifies that a method be defined on the generic function named *s_readerFun* to read the value of the given slot.
`@writer` *s_writerFun*	Specifies that a method be defined on the generic function named *s_writerFun* to change the value of the given slot.
`@initform` *g_exp*	The expression is evaluated every time an instance is created. The `@initform` slot option is used to provide an expression to be used as a default initial value for the slot. The form is evaluated in the class definition environment.

Value Returned

`t`	Always returns `t`.

Example 1

```
defclass( Point
    ( GeometricObject )
    (
        ( name @initarg name )
        ( x @initarg x )       ;;; x-coordinate
        ( y @initarg y )       ;;; y-coordinate
        )
    ) ; defclass => t
P = makeInstance( 'Point ?name "P" ?x 3 ?y 4 )
```

findClass

```
findClass(
     s_className
     )
     => u_classObject | nil
```

Description

Returns the class object associated with a symbol. The symbol is the symbolic name of the class object.

Arguments

s_className	A symbol that denotes a class object.

Value Returned

u_classObject	Class object associated with a symbolic name.
nil	If there is no class associated with the given symbol.

Example

```
findClass( 'Point )          => funobj:0x1c9968
findClass( 'fixnum )         => funobj:0x1840d8
findClass( 'standardObject   => funobj:0x184028
findClass( 'fuzzyNumber )    => nil
```

Reference

defclass, className

initializeInstance

```
initializeInstance(
     g_instance
         [ u_?initArg1     value1 ]
         [ u_?initArg2     value2 ] ...
     )
     => t
```

Description

Initializes the newly created instance of a class. `initializeInstance` is called by the `makeInstance` function.

Arguments

g_instance	A symbol denoting an instance. The instance must be created using `makeInstance`.
`u_?initArg1`*value1* `u_?initArg2`*value2*	`initArg1`*value1* is the initial value for argument1 of the instance. Similarly for the pair `u_initArg2`*value2* and so forth.

Value Returned

t	The instance has been initialized.

Example

```
defclass( A () ())
=>t

defmethod( initializeInstance ((obj A) @key (a 3) @rest args))
      (printf "initializeInstance : A : was called with args - obj == '%L'
      a == '%L' rest == '%L'\n" obj a args)

(callNextMethod)
=>t

makeInstance( 'A ?a 7)
      initializeInstance : A : was called with args - obj == 'stdobj@0x83bf048' a
      == '7' rest == 'nil'
=>stdobj@0x83bf048

makeInstance( 'A)
       initializeInstance : A : was called with args - obj == 'stdobj@0x83bf054' a
      == '3' rest == 'nil'
=>stdobj@0x83bf054
```

isClass

```
isClass(
     g_object
     )
     => t | nil
```

Description:

Checks if the given object is a class object.

Arguments

g_object	Any SKILL object.

Value Returned

`t`	If the given object is a class object.
`nil`	Otherwise.

Example

```
isClass( classOf( 5 ) ) => t
isClass( findClass( 'Point ) ) => t
isClass( 'noClass ) => nil
```

Reference

classOf, findClass

makeInstance

```
makeInstance(
     us_class
         [ u_?initArg1    value1 ]
         [ u_?initArg2    value2 ] ...
     )
     => g_instance
```

Description

Creates an instance of a class, which can be given as a symbol or a class object.

Arguments

us_class	Class object or a symbol denoting a class object. The class must be either `standardObject` or a subclass of `standardObject`.
`u_?initArg1` *value1* `u_?initArg2` *value2*	The symbol `u_initArg1` is specified in one of the slot specifiers in the `defclass` declaration of either *us_class* or a superclass of *us_class*. *value1* is the initial value for that slot. Similarly for the pair `u_initArg2` *value2* and so forth.

Value Returned

g_instance	The instance. The print representation of the instance resembles `stdobj:`*xxxxx*, where *xxxxx* is a hexadecimal number.

Example

```
defclass( Circle ( GeometricObject )
    (( center @initarg c ) ( radius @initarg r )) ) => t
P = makeInstance( 'Point ?name "P" ?x 3 ?y 4 )
    => stdobj:0x1d003c
C = makeInstance( 'Circle ?c P ?r 5.0 ) => stdobj:0x1d0048
makeInstance( 'fixnum )
*Error* unknown: non-instantiable class - fixnum
```

Reference

defclass

printself

```
printself(
     g_object
     )
     => g_result
```

Description

A generic function which is called to print a `stdObject` instance.

Arguments

g_object	An instance of a class.

Value Returned

g_result	A string or symbol representing information about `g_object`.

Example

```
defmethod( printself ((obj myClass))
sprintf(nil "#{instance of myClass:%L}" obj) ; returns a string
)

i = makeInstance('myClass)
=> #{instance of myClass:stdobj@0x83ba018}

; prints all instances of myClass
```

setSlotValue

```
setSlotValue(
     g_standardObject
     s_slotName
     g_value
     )
     => g_value
```

Description

Sets the *s_slotName* slot of *g_standardObject* to *g_value*.

An error is signaled if there is no such slot for the *g_standardObject*. This function bypasses any `@writer` generic function for the slot that you specified in the `defclass` declaration for the *g_standardObject*'s class.

Arguments

g_standardObject	An instance of the *standardObject* class or a subclass of *standardObject*.
s_slotName	The slot symbol used as the slot name in the `defclass` slot specification.
g_value	Any SKILL data object.

Value Returned

g_value	The value assigned to the slot.

Example

```
defclass( GeometricObject ()
      (
            ( x @initarg x )
            ( y @initarg y )
      )
)     => t
geom = makeInstance( 'GeometricObject ?x 0 )
                            => stdobj:0x34b018
slotValue( geom 'y )        => \*slotUnbound\*
setSlotValue( geom 'y 2 )   => 2
slotValue( geom 'y )        => 2
```

sharedInitialize

```
sharedInitialize(
     g_object
     g_slotList
     @rest l_initargs
     )
     => g_object | error
```

Description

This is a generic function, which is called when an instance is created, re-initialized, updated to conform to a redefined class, or updated to conform to a different class. It is called from the `initializeInstance`, `updateInstanceForRedefinedClass`, and `updateInstanceForDifferentClass` functions to initialize slots of the instance `g_object` using the corresponding `initforms`.

If the funciton is successful, the updated instance is returned.

Arguments

g_object	An instance of a class.
g_slotList	`t` or a list of slot names (symbols). If the argument is `t`, it initializes all uninitialized slots. If it is a list of slot names, it initializes only the uninitialized slots in the list.
l_initargs	List of optional `initargs`.

Value Returned

g_object	The updated instance (`g_object`).
error	Otherwise.

Example

```
defclass( A () ((a @initform 1)))
=>t

defmethod( sharedInitialize ((obj A) slots @key k @rest args)
(printf "sharedInitialize A: obj->?? == '%L' k == '%L' args == '%L'\n" obj->?? k
args)
(callNextMethod)

)
```

```
=>t

defclass( B () ((b @initform 2)))
=>t

x = makeInstance( 'A ?k 9)

sharedInitialize A: obj->?? == '(a \*slotUnbound\*)' k == '9' args == 'nil'
=>stdobj@0x83bf018

defclass( A () ((a @initform 1)
(c @initform 3)))

*WARNING* (defclass): redefinition of class A updating stdobj@0x83bf018
sharedInitialize A: obj->?? == '(a 1 c \*slotUnbound\*)' k == 'nil' args == 'nil'

=>t

changeClass( x 'B ?k 7)

updating stdobj@0x83bf018

stdobj@0x83bf018

x->??

(b 2)

changeClass( x 'A ?k 7)

updating stdobj@0x83bf018

sharedInitialize A: obj->?? == '(a \*slotUnbound\* c \*slotUnbound\*)' k == '7'
args == 'nil'

stdobj@0x83bf018

x->??

(a 1 c 3)
```

slotBoundp

```
slotBoundp(
     obj
     t_slotName
     )
     => t/nil
```

Description

Checks if a named slot is bound to an instance or not.

Note: For compatibility with previous releases, an alias to this function with the name `ilSlotBoundp` exists.

Arguments

obj	An instance of some class.
t_slotName	Slot name.

Value Returned

t	If the slot is bound.
nil	If the slot is unbound.

Note: It throws an error if `obj` or `t_slotName` is invalid.

Example

```
myObject => slotX = 20
slotBoundp(myObject "slotX") => t
```

slotUnbound

```
slotUnbound(
     u_class
     g_object
     s_slotName
     )
     => g_result
```

Description

This function is called when the `slotValue` function attempts to reference an unbound slot. It signals that the value of the slot `s_slotName` of `g_object` has not been set yet. In this case, `slotValue` returns the result of the method.

Arguments

u_class	A class object. The class must be either `standardObject` or a subclass of `standardObject`.
g_object	An instance of *u_class*.
s_slotName	The name of the unbound slot.

Value Returned

g_value	Value contained in the slot *s_slotName*. The default value is '*slotUnbound*.

Example

```
defclass( A () ((a)))
=>t

x = (makeInstance 'A)
=>stdobj@0x83bf018

defmethod( slotUnbound (class (obj A) slotName) (printf "slotUnbound : slot '%L'is
unbound\n" slotName) (setSlotValue obj slotName 6)
)
=>t

x->a
=>slotUnbound : slot 'a' is unbound
=>6

x->a
=>6
```

```
defmethod( slotUnbound (class (obj A) slotName) (printf "slotUnbound : slot '%L'
is unbound\n" slotName) (setSlotValue obj slotName 6)
8
)
=>t

*WARNING* (defmethod): method redefined generic:slotUnbound class:(t A t)

x->a = '\*slotUnbound\*

\*slotUnbound\*

x->a
=>slotUnbound : slot 'a' is unbound
=>8 ;; the return value of slotUnbound method, not a new value of the slot

x->a
=>6
```

slotValue

```
slotValue(
     g_standardObject
     s_slotName
     )
     => g_value
```

Description

Returns the value contained in the slot *slotName* of the given *standardObject*.

If there is no slot with the given name an error is signalled. This function bypasses any `@reader` generic function for the slot that you specified in the `defclass` declaration for the *g_standardObject*'s class.

Arguments

g_standardObject	An instance of the *standardObject* class or a subclass of *standardObject*.
s_slotName	The slot symbol used as the slot name in the *defclass* slot specification.

Value Returned

g_value	Value contained in the slot *s_slotName* of the given *standardObject*.

Example

```
defclass( GeometricObject ()
      (
            ( x @initarg x )
            ( y @initarg y )
      )
)
=> t

defclass( Point
      ( GeometricObject)
      (
            ( name @initarg name )
      )
)                                => t
```

```
p = makeInstance( 'Point ?name "P" ?x 0 ?y 10 )
                                => stdobj:0x355024
slotValue( p 'name )            => "P"
slotValue( p 'x )               => 0
slotValue( p 'y )               => 10
```

2

Generic Functions and Methods

callAs

```
callAs(
     us_class
     s_genericFunction
     g_arg1
     [ g_arg2 ... ]
     )
     => g_value
```

Description

Calls a method specialized for some super class of the class of a given object directly, bypassing the usual method inheritance and overriding of a generic function.

It is an error if the given arguments do not satisfy the condition `(classp g_obj us_class)`.

Arguments

us_class	A class name or class object.
s_genericFunction	A generic function name.
g_arg1	A SKILL object whose class is *us_class* or a subclass of *us_class*.
g_arg2	Arguments to pass to the generic function.

Value Returned

g_value	The result of applying the selected method to the given arguments.

Example

```
defclass( GeometricObj () ())
=> t
defclass( Point (GeometricObj ) () )
=> t
defgeneric( whoami (obj) println("default"))
=> t
defmethod( whoami  (( obj Point )) println("Point"))
=> t
defmethod( whoami  (( obj GeometricObj))
                     println( "GeometricObj"))
=> t
p = makeInstance( 'Point )
=> stdobj:0x325018
whoami(p)                                  ;prints "Point"
=> nil
callAs( 'GeometricObj 'whoami p )          ;prints "GeometricObj"
=> nil
```

Reference

nextMethodp, callNextMethod

callNextMethod

```
callNextMethod(
     [ g_arg ... ]
     )
     => g_value
```

Description

Calls the next applicable method for a generic function from within the current method. Returns the value returned by the method it calls.

This function can only be (meaningfully) used in a method body to call the next more general method in the same generic function.

You can call `callNextMethod` with no arguments, in which case all the arguments passed to the calling method will be passed to the next method. If arguments are given, they will be passed to the next method instead.

Arguments

g_arg	Optional arguments to pass to the next method.

Value Returned

g_value	Returns the value returned by the method it calls.

Example

If you call the `callNextMethod` function outside a method you get:

```
ILS-<2> procedure( example() callNextMethod() )
example
ILS-<2> example()
*Error* callNextMethod: not in the scope of any generic function call
```

This example also shows the effect of incrementally defining methods:

```
ILS-<2> defgeneric( HelloWorld ( obj )
        printf( "Generic Hello World\n" )
        )
=> t
ILS-<2> HelloWorld( 5 )
Generic Hello World
=> t
; t is the superclass of all classes
```

```
ILS-<2> defmethod( HelloWorld ((obj t ))
        printf( "Class: %s says Hello World\n" 't )
        )
=> t
ILS-<2> HelloWorld( 5 )
Class: t says Hello World
=> t
; systemObject is a subclass of t
ILS-<2> defmethod( HelloWorld ((obj systemObject  ))
        printf( "Class: %s says Hello World\n" 'systemObject )
        callNextMethod()
        )
=> t
ILS-<2> HelloWorld( 5 )
Class: systemObject says Hello World
Class: t says Hello World
=> t
; primitiveObject is a subclass of systemObject
ILS-<2> defmethod( HelloWorld (( obj primitiveObject ))
        printf( "Class: %s says Hello World\n" 'primitiveObject )
        callNextMethod()
        )
=> t
ILS-<2> HelloWorld( 5 )
Class: primitiveObject says Hello World
Class: systemObject says Hello World
Class: t says Hello World
=> t
; fixnum is a subclass of primitiveObject
ILS-<2> defmethod( HelloWorld (( obj fixnum ))
        printf( "Class: %s says Hello World\n" 'fixnum )
        callNextMethod()
        )
=> t
ILS-<2> HelloWorld( 5 )
Class: fixnum says Hello World
Class: primitiveObject says Hello World
Class: systemObject says Hello World
Class: t says Hello World
=> t
ILS-<2> HelloWorld( "abc" )
Class: primitiveObject says Hello World
Class: systemObject says Hello World
Class: t says Hello World
=> t
```

Reference

nextMethodp, callAs

defgeneric

```
defgeneric(
     s_functionName
         ( s_arg1
         [ s_arg2 ... ]
        )
         [ g_exp ... ]
     )
     => t
```

Description

Defines a generic function with an optional default method. This is a macro form. Be sure to leave a space after the function name. See the *Cadence SKILL Language User Guide* for a discussion of generic functions.

Arguments

s_functionName	Name of the generic function. Be sure to leave a space after the function name.
s_arg1	Any valid argument specification for SKILL functions, including `@key`, `@rest`, and so forth.
g_exp	The expressions that compose the default method. The default method is specialized on the class `t` for the first argument. Because all SKILL objects belong to class `t`, this represents the most general method of the generic function and is applicable to any argument.

Value Returned

`t`	Generic function is defined.

Example

```
ILS-<2> defgeneric( whatis ( object )
            printf(
            "%L is an instance of %s\n"
            object className( classOf( object))
                )
            ) ; defgeneric
ILS-<2> whatis( 5 )
5 is an instance of fixnum
```

```
t
ILS-<2> whatis( "abc" )
"abc" is an instance of string
t
```

Reference

defmethod

defmethod

```
defmethod(
     s_name
         (
         ( s_arg1
         s_class
         )
         s_arg2 ...
         )
         g_exp1 ...
     )
     => t
```

Description

Defines a method for a given generic function. This is a macro form. Be sure to leave a space after *s_name*.

The method is specialized on the *s_class*. The method is applicable when `classp( s_arg1 s_class )` is true.

Arguments

s_name	Name of the generic function for which this method is to be added. Be sure to leave a space after *s_name*.
`(` *s_arg1 s_class*`)`	List composed of the first argument and a symbol denoting the class. The method is applicable when *s_arg1* is bound to an instance of *s_class* or one of its subclasses.
g_exp1 ...	Expressions that compose the method body.

Value Returned

`t`	Always returns `t`.

Example

```
defmethod( whatis (( p Point ))
    sprintf( nil "%s %s @ %n:%n"
        className( classOf( p ))
        p->name
        p->x
        p->y
```

```
        )
    ) ; defmethod
=> t
```

Reference

defgeneric, procedure, defun

getMethodSpecializers

```
getMethodSpecializers(
     s_genericFunction
     )
     => l_classNames | nil
```

Description

Returns the specializers of all methods currently associated with the given generic function, in a list of class names. The first element in the list is `t` if there is a default method.

Arguments

s_genericFunction	A symbol that denotes a generic function object.

Value Returned

l_classNames	List of method specializers that are currently associated with *s_genericFunction*. The first element in the list is `t` if there is a default method.
`nil`	*s_genericFunction* is not a generic function.

Example

```
defmethod( met1 ((obj number)) println(obj))
=> t

getMethodSpecializers('met1)
=>(number)

defclass( XGeometricObj () () )
=> t

defgeneric( whoami (obj) printf("Generic Object\n"))
=> t

defmethod( whoami (( obj XGeometricObj)) printf( "XGeometricObj, which is also
a\n"))
=> t

getMethodSpecializers('whoami)
=> (t XGeometricObj)

getMethodSpecializers('car)
=> *Error* getMethodSpecializers: first argument must be a generic function - car
nil

getMethodSpecializers(2)
=> *Error* getMethodSpecializers: argument #1 should be a symbol (type template =
"s") - 2
```

getGFbyClass

```
getGFbyClass(
     s_className
     [g_nonExistent]
     )
     => l_methods
```

Description

Displays the list of all generic functions specializing on a given class.

Arguments

s_className	Name of the class for which you want view the list of specializing functions.
g_nonExistent	If set to t, lists the list of generic functions specializing on non-defined classes only.

Value Returned

l_methods	A list of generic functions.

Example

```
getGFbyClass('systemObject)
  => (printObject)
```

getApplicableMethods

```
getApplicableMethods(
     s_gfName
     l_args
     )
     =>l_funObjects
```

Description

Returns a list of applicable methods (*funObjects*) for the specified generic function for a given set of arguments. The returned list contains methods in the calling order.

Arguments

s_gfName	Specifies the name of the generic function
l_args	Specifies a list of arguments for which you want to retrieve the applicable methods

Values Returned

l_funObjects	Returns a list of methods in the calling order **Note:** If there are no applicable methods for the given arguments then an error is raised.

Example

```
getApplicableMethods('testMethod list("test" 42))
=> (funobj@0x83b76d8 funobj@0x83b76f0 funobj@0x83b76a8 funobj@0x83b7678
funobj@0x83b7690 funobj@0x83b7630 funobj@0x83b7600 funobj@0x83b76c0 )
```

getMethodName

```
getMethodName(
     U_funObject
     )
     =>s_name
```

Description

Returns the method name for the given function object

Arguments

U_funObject	Specifies the name of the function object for which you want to retireve the method name

Values Returned

s_name	Returns the method name for the specified generic function object

Example

```
getMethodName(funobj@0x0182456)
=> testMethod
```

getMethodRole

```
getMethodRole(
     U_funObject
     )
     =>s_role / nil
```

Description

Returns the method role for the given function object. *U_funObject* should be a valid generic function object.

Arguments

U_funObject	Specifies the name of the function object for which you want to retireve the method role. This should be a valid generic function object.

Values Returned

s_role	Returns the role of the specified generic function object
nil	Returns nil if the method is a primary method

Example

```
getMethodRole(funobj@0x0182456)
=> @before
```

getMethodSpec

```
getMethodSpec(
     U_funObject
     )
     =>l_spec
```

Description

Returns the list of specializer for the given funobject. *U_funObject* should be a valid generic method object.

Arguments

U_funObject	Specifies the name of the function object for which you want to retrieve the list of specializers. This should be a valid generic method object.

Values Returned

l_spec	Returns a list of specializers for the specified generic function object

Example

```
getMethodSpec(funobj@0x0182456)
=> (string number)
```

getGFproxy

```
getGFproxy(
     s_gfName
     )
     => U_classObj / nil
```

Description

Returns a *proxy* instance from the specified generic function object

Arguments

s_gfName	Specifies a symbol that denotes the name of a generic function object

Value Returned

U_classObj	Returns the associated proxy instance
nil	Returns nil if a generic function does not exist

Example

```
getGFproxy('niTest); niTest is the name of the generic function
  => stdobj@0x83c0018
classOf(getGFproxy('niTest))
  => class:niGF
classOf(getGFproxy('printself))  ;; class of standard generic function (printself)
  => class:ilGenericFunction  ;; default
getGFproxy('abc)
  => nil ;; non-existing generic function
```

nextMethodp

```
nextMethodp(
     )
     => t | nil
```

Description

Checks if there is a next applicable method for the current method's generic function. The *current method* is the method that is calling `nextMethodp`.

`nextMethodp` is a predicate function which returns `t` if there is a next applicable method for the current method's generic function. This next method is specialized on a superclass of the class on which the current method is specialized.

Prerequisites

This function should only be used within the body of a method to determine whether a next method exists.

The return value and the effect of this function are unspecified if called outside of a method body.

Arguments

None.

Value Returned

`t`	There is a next method
`nil`	There is no next method.

Example

```
defclass( GeometricObj () ())
=> t
defclass( Point ( GeometricObj ) () )
=> t
defmethod( whoami (( obj Point ))
            if( nextMethodp()
```

```
            then printf("Point, which is a " )
            callNextMethod()
            else printf("Point")))
=> t
p = makeInstance( 'Point )
=> stdobj:0x325030
whoami(p)
=> t
```

Prints `Point.`

```
defmethod( whoami  (( obj GeometricObj))
                println( "GeometricObj"))
=> t
whoami( p)
=> nil
```

Prints `Point, which is a "GeometricObj"`

Reference

defmethod, callNextMethod

removeMethod

```
removeMethod(
     s_genFunction
     g_className
     [g_method]
     )
     => t/nil
```

Description

Removes a given method from a generic function.

Note: For compatibility with previous releases, an alias to this function with the name, `ilRemoveMethod` exists.

Arguments

s_genFunction	Name of the generic function from which the method needs to be removed.
g_className	Name of the class or list of classes to which the generic function belongs.
g_method	Specifies the method qualifier. It can have one of the following values: '@before, '@after, and '@around. If this value is not provided or is specified as `nil`, then the primary method is removed.

Value Returned

`t`	Returns `t`, if the method is successfully removed.
`nil`	Returns `nil`, if the method is not removed.

Example

```
removeMethod('my_function 'my_class '@before)

removeMethod('myFunB '(classX classY) '@after)
```

updateInstanceForDifferentClass

```
updateInstanceForDifferentClass(
     g_previousObj
     g_currentObj
     @rest initargs
     )
     => t
```

Description

A generic function, which is called from `changeClass` to update the specified instance (`g_currentObj`).

Arguments

g_previousObj	A copy of the `ilChangeClass` argument. It keeps the old slot values of the specified instance.
g_currentObj	The instance to be updated.
initargs	Additional arguments for the instance.

Value Returned

`t`	Always returns `t`

updateInstanceForRedefinedClass

```
updateInstanceForRedefinedClass(
     obj
     l_addedSlots
     l_deletedSlots
     l_dplList
     )
     => t
```

Description

It is a generic function, which is called to update all instances of a class, when a class redefinition occurs.

The primary method of `updateInstanceForRedefinedClass` checks the validity of `initargs` and throws an error if the provided `initarg` is not declared. It then initializes slots with values according to the `initargs`, and initializes the newly added-slots with values according to their `initform` forms.

When a class is redefined and an instance is being updated, a property-list is created that captures the slot names and values of all the discarded slots with values in the original instance. The structure of the instance is transformed so that it conforms to the current class definition.

The arguments of `updateInstanceForRedefinedClass` are the transformed instance, a list of slots added to the instance, a list of slots deleted from the instance, and the property list containing the slot names and values of slots that were discarded. This list of discarded slots contains slots that were local in the old class and are shared in the new class.

Arguments

obj	Instance of the class being redefined.
l_addedSlots	A list of slots added to the class.
l_deletedSlots	A list of slots deleted from the class.
l_dplList	A list of slots that were discarded, with their values.

Value Returned

t	Always returns t

Example

Define a method for the class myClass (to be applied to all instances of myClass if it is redefined):

```
(defmethod updateInstanceForRedefinedClass ((obj myClass) added deleted
dplList @rest initargs)
;;callNextMethod for obj and pass ?arg "myArg" value for slot arg
(apply callNextMethod obj added deleted dplList ?arg "myArg" initargs)
)
```

3

Generic Specializers

ilArgMatchesSpecializer

```
ilArgMatchesSpecializer(
     U_genericfuncObj
     s_specClass
     s_specArg)
     => t/nil
```

Description

Checks if the given argument matches generic specializer

Arguments

U_genericfuncObj	Specifies a generic function object
s_specClass	Specifies the generic specializer class
s_specArg	Specifies the specializer argument

Value Returned

t	Returns `t` if the given argument matches
`nil`	Returns `nil` if the given argument does not match

Example

```
(defmethod ilArgMatchesSpecializer (gf (theClass mySpec) arg)
(eq arg->type 'polygon))
```

ilEquivalentSpecializers

```
ilEquivalentSpecializers(
     U_genericfuncObj
     s_spec1
     s_spec2)
     => t/nil
```

Description

Defines a method to check if two specializers are equal (required during method redefinition).

Arguments

U_genericfuncObj	Specifies the generic function object
s_spec1	Specifies the first specializer
s_spec2	Specifies the second specializer

Value Returned

`t`	Returns `t` if the two specializers are equal
`nil`	Returns `nil` if the two specializers are not equal

Example

```
(defmethod ilEquivalentSpecializers(gf spec1 spec2)
classOf(spec1) == classOf(spec2))
```

ilGenerateSpecializer

```
ilGenerateSpecializer(
     U_genericfuncObj
     s_specClass
     s_specArg)
     => g_expression
```

Description

Returns a SKILL expression that makes an instance of the given specializer class and optionally set the slots. In the generated SKILL expression, *s_specArg* can be used to initialize the slots.

Arguments

U_genericfuncObj	Specifies the generic function object
s_specClass	Specifies the generic specializer class
s_specArgs	Specifies the evaluated specializer arguments that are defined in `defmethod`

Value Returned

g_expression	Returns a SKILL expression that is to be evaluated inside `defmethod`

Example

```
; create an instance without any slot
(defmethod ilGenerateSpecializer (gf (specName t) specArgs)
    `(makeInstance ',specName)
```

ilSpecMoreSpecificp

```
ilSpecMoreSpecificp(
     U_genericfuncObj
     s_spec1
     s_spec2
     s_specArg)
     => t / nil
```

Description

Checks if spec1is more specific than spec2. You need to define all required `ilSpecMoreSpecificp` methods for all existing custom specializers (so that the system can find a method to compare any pair of custom specializers).

Arguments

U_genericfuncObj	Specifies a generic function object
s_spec1	Specifies the first generic specializer class
s_spec2	Specifies the second generic specializer class
s_specArg	Specifies the specializer argument

Value Returned

`t`	Returns `t` if spec1 is more specific than spec2
`nil`	Returns `nil` to indicate that the custom specializer definition is inconsistent

Example

```
(defmethod ilSpecMoreSpecificp (gf (spec1 classSpec1) (spec2 classSpec2)
args)spec1->value > spec2->value)
```

4

Subclasses and Superclasses

subclassesOf

```
subclassesOf(
     u_classObject
     )
     => l_subClasses
```

Description

Returns the ordered list of all (immediate) subclasses of *u_classObject*. Each element in the list is a class object.

The list is sorted so that each element of the list is a subclass of the remaining elements.

Arguments

u_classObject	A class object.

Value Returned

l_subClasses	The list of subclasses. If the argument is not a class object, then *l_subClasses* is `nil`.

Example

```
L = superclassesOf( findClass( 'fixnum ))
subclassesOf( findClass( ’primitiveObject ) )
      => (class:list class:port class:funobj class:array class:string
            class:symbol class:number
         )

      subclassesOf( 5 ) => nil
```

subclassp

```
subclassp(
     u_classObject1
     u_classObject2
     )
     => t | nil
```

Description

Predicate function that checks if *classObject1* is a subclass of *classObject2*.

A class *C1* is a subclass of class *C2* if *C2* is a (direct or indirect) superclass of *C1*.

Arguments

u_classObject1	A class object.
u_classObject2	A class object.

Value Returned

t / nil	*s_class2* is a (direct or indirect) superclass of *s_class1*.

Example

```
subclassp( findClass( 'Point ) findClass( 'standardObject )) => t
subclassp(
    findClass( 'fixnum )
    findClass( 'primitiveObject ))
=> t
subclassp(
    findClass( 'standardObject )
    findClass( 'primitiveObject )
    )
=> nil
```

Reference

superclassesOf

superclassesOf

```
superclassesOf(
     u_classObject
     )
     => l_superClasses
```

Description

Returns the ordered list of all super classes of `u_classObject`. Each element in the list is a class object.

The list is sorted so that each element of the list is a subclass of the remaining elements.

Note: If a class is inherited from multiple classes, `superclassesOf()` traverses the entire inheritance tree and returns the linearized class list.

Arguments

u_classObject	A class object.

Value Returned

l_superClasses	The list of super classes. If the argument is not a class object, then *l_superClasses* is `nil`.

Example

```
defclass(basicA () ())
defclass(basicB () ())
defclass(derived1 (basicA) ())
defclass(derived2 (basicA basicB) ())
superclassesOf(findClass('derived1))
=> (class:derived1 class:basicA class:standardObject class:t)
superclassesOf(findClass('derived2))
=> (class:derived2 class:basicA class:basicB class:standardObject class:t)
```

5

Dependency Maintenance Protocol Functions

The dependency maintenance protocol provides a way to register an object that is notified whenever a class or generic function on which it is set is modified. The registered object is called a *dependent* of the class or generic function metaobject. SKILL uses the `addDependent` and `removeDependent` methods to maintain the dependents of a class or a generic function metaobject. The dependents can be accessed using the `getDependents` method. The dependents are notified about a modified class or generic function by calling the `updateDependent` method.

These methods are described in the following section.

addDependent

```
addDependent(
     g_object
     g_dependent
     )
     => t | nil
```

Description

Registers a dependent object for given object. SKILL checks if *g_dependent* already exists as a dependent of *g_object* (using the `eqv` operator), then *g_dependent* is not registered again and `nil` is returned.

Arguments

g_object	Specifies a SKILL object, which could be a class or a generic function, on which the dependent object needs to be set
g_dependent	Specifies the dependent object that you want to set on the given object

Value Returned

`t`	Returns `t` if the dependent object can be successfully set
`nil`	Returns `nil` if the dependent object is already registered for the given object

Example 1

```
addDependent( findClass('class) 'dep1)
```

This example registers the dependent object, `dep1`, for an object of class, `class`.

getDependents

```
getDependents(
     g_object
     )
     => l_dependents
```

Description

Returns a list of dependents registered for the given SKILL object, which could be a class or a generic function

Arguments

g_object	Specifies the SKILL object, which could be a class or a generic function, on which the dependent object needs to be added

Value Returned

l_dependents	Returns a list of dependents registered for the given object

Example 1

```
getDependents( findClass('class))
=> (dep1 dep2)
```

removeDependent

```
removeDependent(
     g_object
     g_dependent
     )
     => t | nil
```

Description

Removes a dependent object from the given object.

Note: An object can be a dependent of multiple SKILL meta objects. If an attempt is made to remove an object from a given meta object of which the object is not a dependent, `removeDependent` will return `nil` but not display any error.

Arguments

g_object	Specifies a SKILL object, which could be a class or a generic function, from which the dependent object needs to be removed
g_dependent	Specifies the dependent object that needs to be removed

Value Returned

`t`	Returns `t` if the dependent object is successfully removed
`nil`	Returns `nil` if the dependent object is not removed

Example 1

```
removeDependent( findClass('class) 'dep1)
```

This example removes the dependent object, `dep1`, from the object of class, `class`.

updateDependent

```
updateDependent(
     u_class
     g_dependent
     s_notifType
     u_classObj
     )
     => t
```

Description

Updates the dependents of a SKILL object, which could be a class or a generic function, when the SKILL object is modified. The SKILL engine calls this method for each *g_dependent* at different times. For example, if *g_dependent* is a method, the SKILL engine calls `updateDependent` at the time of adding or removing the method; whereas, for dependent classes the SKILL engine calls the `updateDependent` method at the end of class creation.

Note: Your applications can implement methods on this generic function.

Arguments

u_class	Specifies a SKILL object, which could be a generic function or a class, for which the dependents need to be updated. Depending on the SKILL object specified, different arguments are passed. For example, in case the specified SKILL object is a generic function then the dependent object could be a generic function object or a *proxy object* and in case of the SKILL object is a class, then `class:class` can be specified as the dependent object.
g_dependent	Specifies a dependent object
s_notifType	Specifies the type of update that has occurred using the following qualifiers `add_method`, `remove_method`, `add_class`, `redef_class`, `add_generic`, and `redef_generic`.
u_classObj	Specifies the class object when a new class is defined. This argument is `nil` when a class is redefined.

Value Returned

The return value is ignored.

Example 1

```
defmethod( ilUpdateDependent((proxy class) obj dep type)
  printf("updateDependent called for CLASS -- %L" classOf(proxy))
  printf(" obj : %L type : %L\n" obj type)
  printf("Dependents : %L\n" get(className(proxy) '\*dependents\*))
  printf("Dependent : %L\n" dep)
  t
)
```